Implementation

Control Engineering (Reguleringsteknik)

Christoffer Sloth

chsl@mmmi.sdu.dk

SDU Robotics
The Maersk Mc-Kinney Moller Institute
University of Southern Denmark

Agenda

Introduction
- Curriculum

Dynamical Systems
- Third and Higher Order Systems
- Zeros

Digitization
- Emulation
- Numerical Integration Methods
- Discrete Design

Summary

Introduction
Curriculum for Reguleringsteknik (REG)

Matematiske og grafiske metoder til syntese af lineære tidsinvariante systemer:[1]

- diskret og kontinuert tilstandsbeskrivelse
- analyse i tid og frekvens
- stabilitet, reguleringshastighed, følsomhed og fejl
- digitale PI, PID, LEAD og LAG regulatorer (serieregulatorer)
- tilstandsregulering, pole-placement og tilstands-estimering (observer)
- optimal regulering (least squares) og optimal tilstands-estimation (Kalman-filter)

Færdigheder:
Efter gennemførelse af kurset kan den succesfulde studerende:

- kunne analysere, dimensionere og implementere såvel kontinuert som tidsdiskret regulering af lineære tidsinvariante og stokastiske systemer

Kompetencer:
Efter gennemførelse af kurset kan den succesfulde studerende:

- anvende og implementere klassiske og moderne reguleringsteknikker for at kunne styre og regulere en robot hurtig og præcist

[1] Based on https://fagbesk.sam.sdu.dk/?fag_id=39673

Introduction
Course Overview

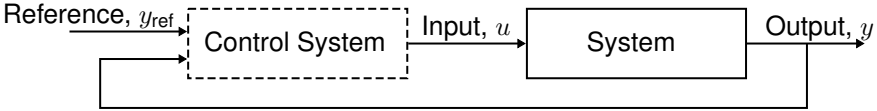
The twelve lectures of the course are

- **Lecture 1**: Introduction to Linear Time-Invariant Systems
- **Lecture 2**: Stability and Performance Analysis
- **Lecture 3**: Introduction to Control
- **Lecture 4**: Design of PID Controllers
- **Lecture 5**: Root Locus
- **Lecture 6**: The Nyquist Plot
- **Lecture 7**: Dynamic Compensators and Stability Margins
- **Lecture 8**: Implementation
- **Lecture 9**: State Feedback
- **Lecture 10**: Observer Design
- **Lecture 11**: Optimal Control (Linear Quadratic Control)
- **Lecture 12**: The Kalman Filter

Introduction

Motivating Example

Task: Design a cruise control for a car.

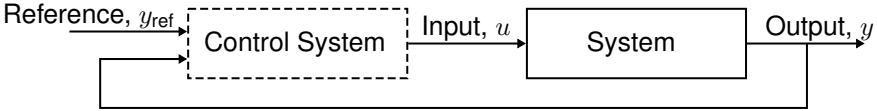


- **Control Input**: Throttle position u
- **Measured Output**: Velocity of the car y
- **Reference Input**: Desired velocity of the car y_{ref}

Introduction

Motivating Example

Task: Design a cruise control for a car.



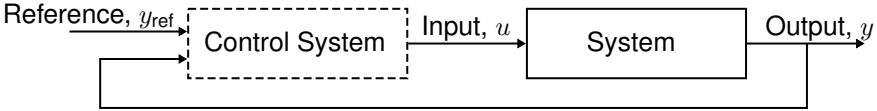
Today, an answer to the following question is provided:

1. How can one ensure that a control is stable despite uncertainties in the system?

Introduction

Motivating Example

Task: Design a cruise control for a car.



Today, an answer to the following question is provided:

1. How can one ensure that a control is stable despite uncertainties in the system?
   - Uncertainties may affect the *gain* of the system (e.g. the inertia).
   - Uncertainties may affect the *phase* of the system (e.g. communication delays between controller and sensor).

Dynamical Systems

Introduction
Curriculum

Dynamical Systems
Third and Higher Order Systems
Zeros

Digitization
Emulation
Numerical Integration Methods
Discrete Design

Summary

Third and Higher Order Systems

Introduction

We have previously analyzed 1st order and 2nd order systems (with unity gain)

$$G_1(s) = \frac{1}{\tau s + 1} \qquad G_2(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

Third and Higher Order Systems

Introduction

We have previously analyzed 1st order and 2nd order systems (with unity gain)

$$G_1(s) = \frac{1}{\tau s + 1} \qquad G_2(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

A general transfer function

$$G(s) = \frac{b_m(s - z_1)(s - z_2)\cdots(s - z_m)}{a_m(s - p_1)(s - p_2)\cdots(s - p_n)}$$

can be written as a product of first order systems (real poles) and second order systems (complex poles).

Third and Higher Order Systems

Third Order System

A third order system with a complex pole-pair can be written as

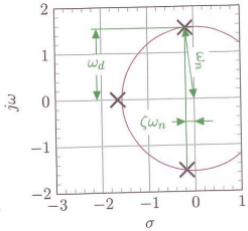
$$G_3(s) = \frac{\omega_b \omega_n^2}{(s + \omega_b)(s^2 + 2\zeta\omega_n s + \omega_n^2)}$$

Third and Higher Order Systems

Third Order System (Example)

A third order system with a complex pole-pair can be written as

$$G_3(s) = \frac{1.65 \cdot 2.42}{(s+1.65)(s^2+0.35s+2.42)}$$

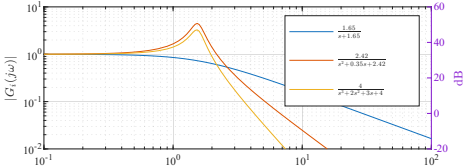


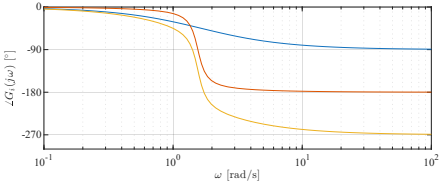
Third and Higher Order Systems

Third Order System (Example)

A third order system with a complex pole-pair can be written as

$$G_3(s) = \frac{1.65 \cdot 2.42}{(s+1.65)(s^2+0.35s+2.42)}$$





Effect of Zeros

Introduction

A general transfer function

$$G(s) = \frac{b_m s^m + \cdots + b_1 s + b_0}{a_n s^n + \cdots + a_1 s + a_0}$$

can be written as

$$G(s) = G_1(s) G_2(s)$$

where

$$G_1(s) = \frac{u_2(s)}{u(s)} = \frac{1}{a_n s^n + \cdots + a_1 s + a_0}$$

$$G_2(s) = \frac{y(s)}{u_2(s)} = b_m s^m + \cdots + b_1 s + b_0$$

Effect of Zeros

Introduction

A general transfer function

$$G(s) = \frac{b_m s^m + \cdots + b_1 s + b_0}{a_n s^n + \cdots + a_1 s + a_0}$$

can be written as

$$G(s) = G_1(s) G_2(s)$$

where

$$G_1(s) = \frac{u_2(s)}{u(s)} = \frac{1}{a_n s^n + \cdots + a_1 s + a_0}$$

$$G_2(s) = \frac{y(s)}{u_2(s)} = b_m s^m + \cdots + b_1 s + b_0$$

The signal $u_2(s)$ is a preconditioned version of $u(s)$ only affected by the system poles.

Effect of Zeros

Introduction

The effect of $G_2(s)$ given by

$$G_2(s) = \frac{y(s)}{u_2(s)} = b_m s^m + \cdots + b_1 s + b_0$$

is to take the time derivatives of u_2 as

$$y(t) = b_m \frac{d^m u_2(t)}{dt^m} + \cdots + b_1 \frac{du_2(t)}{dt} + b_0 u_2(t)$$

This is why only proper transfer functions are considered ($n \geq m$).

Effect of Zeros

Exercise

Compare the impulse responses of

$$G_1(s) = \frac{1}{s+a} \qquad \text{and} \qquad G_2(s) = \frac{s}{s+a} \qquad \text{and} \qquad G_3(s) = \frac{s+1}{s+a}$$

by computing the responses in time-domain.

Effect of Zeros

Non-minimum phase

Zeros in the right half-plane give rise to non-minimum phase behavior.

Effect of Zeros

Non-minimum phase

Zeros in the right half-plane give rise to non-minimum phase behavior.

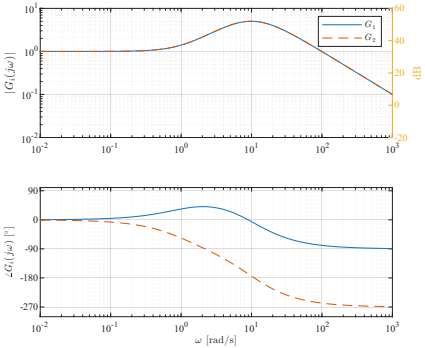
How do bode plots of $G_1(s) = s + 1$ and $G_2(s) = -s + 1$ look like?

Effect of Zeros

Example of Non-minimum phase system

Consider systems

$$G_1(s) = \frac{s+1}{(0.1s+1)^2} \qquad \text{and} \qquad G_2(s) = \frac{-s+1}{(0.1s+1)^2}$$

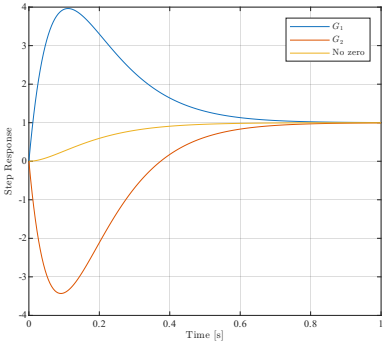


Effect of Zeros

Example of Non-minimum phase system

Consider systems

$$G_1(s) = \frac{s+1}{(0.1s+1)^2} \qquad \text{and} \qquad G_2(s) = \frac{-s+1}{(0.1s+1)^2}$$



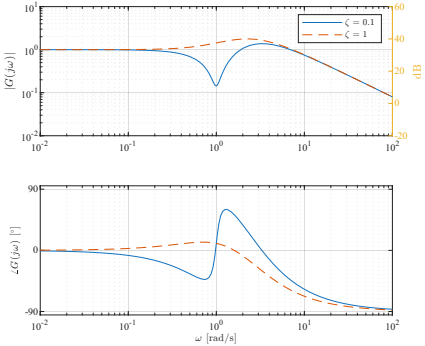
Effect of Zeros

Second-order zero

Higher-order zeros also occur in transfer functions. As an example, consider

$$G(s) = \omega_b^3 \frac{s^2 + 2\zeta\omega_n s + \omega_n^2}{(s+\omega_b)^3}$$

with $\omega_n = 1$, and $\omega_b = 2$.



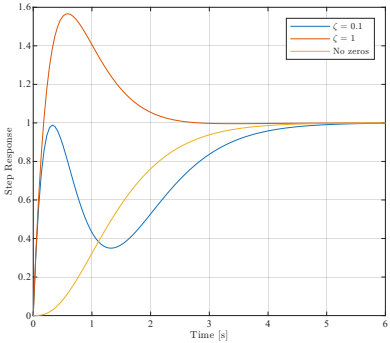
Effect of Zeros

Second-order zero

Higher-order zeros also occur in transfer functions. As an example, consider

$$G(s) = \omega_b^3 \frac{s^2 + 2\zeta\omega_n s + \omega_n^2}{(s + \omega_b)^3}$$

with $\omega_n = 1$, and $\omega_b = 2$.



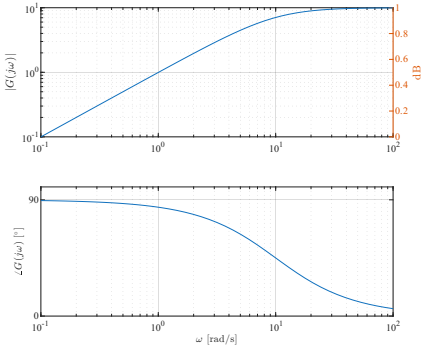
Effect of Zeros

Start Value Theorem

When considering transfer functions with zeros, it is convenient to use the start value theorem that is

$$y(t=0) = \lim_{s\to\infty} sG(s)$$

As an example, consider system $G(s) = \frac{s}{0.1s+1}$.



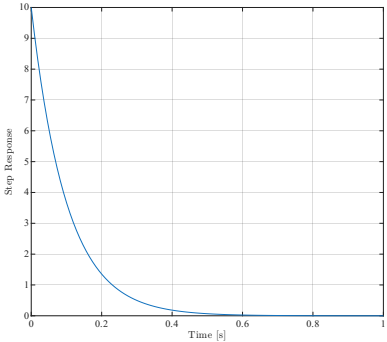
Effect of Zeros

Start Value Theorem

When considering transfer functions with zeros, it is convenient to use the start value theorem that is

$$y(t=0) = \lim_{s\to\infty} sG(s)$$

As an example, consider system $G(s) = \frac{s}{0.1s+1}$.



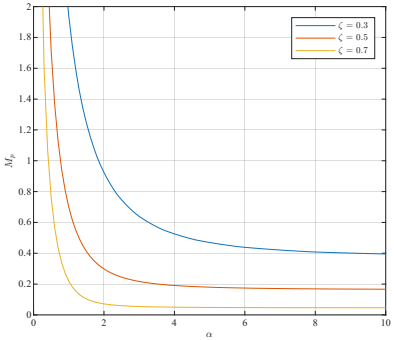
Effect of Zeros

Performance

Consider the transfer function

$$H(s) = \frac{s/(\alpha\zeta) + 1}{s^2 + 2\zeta s + 1}$$

(large α moves zero far away from the origin)



Digitization

Introduction
Curriculum

Dynamical Systems
Third and Higher Order Systems
Zeros

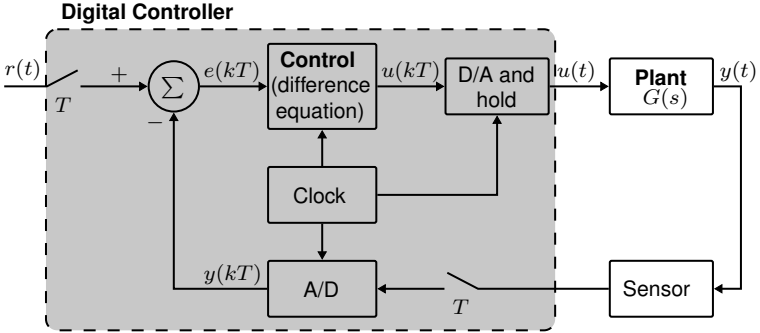
Digitization
Emulation
Numerical Integration Methods
Discrete Design

Summary

Digitization

Introduction

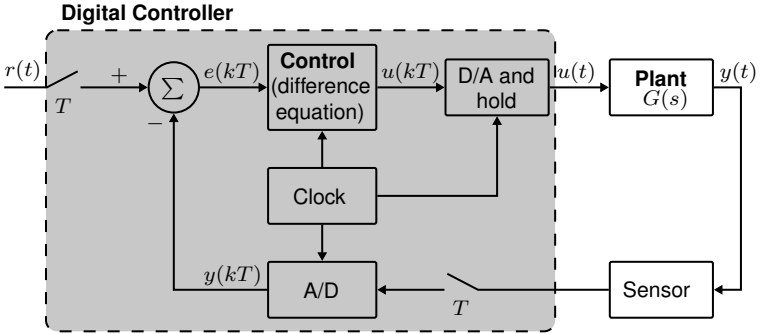
Most control systems are sample data systems, i.e., they consist of both discrete and continuous signals.



Digitization

Introduction

Most control systems are sample data systems, i.e., they consist of both discrete and continuous signals.



A ***digital controller*** both sample and quantize signals. In this course the quantization is omitted; thus, ***discrete controllers*** are designed.

Digitization
Overview

There are two approaches to designing discrete controllers

1. **Emulation**: Design continuous controller $K(s)$ and approximate it with $K(z)$ obtained via e.g. Tustin's method.
2. **Discrete design**: Design the discrete controller directly, without computing $K(s)$.

Digitization
Overview

There are two approaches to designing discrete controllers

1. **Emulation**: Design continuous controller $K(s)$ and approximate it with $K(z)$ obtained via e.g. Tustin's method.
2. **Discrete design**: Design the discrete controller directly, without computing $K(s)$.

The sampling frequency used for the discrete controller should be about 20 times the closed-loop bandwidth. Otherwise, special care should be takes in the design of the discrete controller.

Introduction
Curriculum

Dynamical Systems
Third and Higher Order Systems
Zeros

Digitization
Emulation
Numerical Integration Methods
Discrete Design

Summary

Digitization

Emulation: The PID Controller (1)

Consider the PID controller is s-domain

$$u(s) = \left(k_P + \frac{k_I}{s} + k_D s\right) e(s)$$

and its equivalent in time-domain

$$u(t) = k_P e(t) + k_I \int_0^t e(\tau) d\tau + k_D \frac{d}{dt} e(t)$$

Digitization

Emulation: The PID Controller (1)

Consider the PID controller is s-domain

$$u(s) = \left(k_P + \frac{k_I}{s} + k_D s\right) e(s)$$

and its equivalent in time-domain

$$u(t) = k_P e(t) + k_I \int_0^t e(\tau) d\tau + k_D \frac{d}{dt} e(t)$$

$$u(t) = u_P + u_I + u_D$$

Digitization

Emulation: The PID Controller (1)

Consider the PID controller is s-domain

$$u(s) = \left(k_P + \frac{k_I}{s} + k_D s\right) e(s)$$

and its equivalent in time-domain

$$u(t) = k_P e(t) + k_I \int_0^t e(\tau) d\tau + k_D \frac{d}{dt} e(t)$$
$$u(t) = u_P + u_I + u_D$$

Each of the terms in the PID Controller is computed for discrete times in the following

Digitization

Emulation: The PID Controller (2)

The *proportional term* at time $kT + T$ is

$$u_P(kT + T) = k_P e(kT + T)$$

Digitization

Emulation: The PID Controller (2)

The *proportional term* at time $kT + T$ is

$$u_P(kT + T) = k_P e(kT + T)$$

The *integral term* is

$$u_I(kT + T) = k_I \int_0^{kT+T} e(\tau) d\tau$$

Digitization

Emulation: The PID Controller (2)

The *proportional term* at time $kT + T$ is

$$u_P(kT + T) = k_P e(kT + T)$$

The *integral term* is

$$u_I(kT + T) = k_I \int_0^{kT+T} e(\tau) d\tau$$

$$u_I(kT + T) = \underbrace{k_I \int_0^{kT} e(\tau) d\tau}_{u_I(kT)} + k_I \int_{kT}^{kT+T} e(\tau) d\tau$$

Digitization

Emulation: The PID Controller (2)

The *proportional term* at time $kT + T$ is

$$u_P(kT + T) = k_P e(kT + T)$$

The *integral term* is

$$u_I(kT + T) = k_I \int_0^{kT+T} e(\tau) d\tau$$

$$u_I(kT + T) = \underbrace{k_I \int_0^{kT} e(\tau) d\tau}_{u_I(kT)} + k_I \int_{kT}^{kT+T} e(\tau) d\tau$$

$$u_I(kT + T) \approx u_I(kT) + k_I \frac{T}{2} \left(e(kT + T) + e(kT)\right)$$

Digitization

Emulation: The PID Controller (2)

The *proportional term* at time $kT + T$ is

$$u_P(kT + T) = k_P e(kT + T)$$

The *integral term* is

$$u_I(kT + T) = k_I \int_0^{kT+T} e(\tau) d\tau$$

$$u_I(kT + T) = \underbrace{k_I \int_0^{kT} e(\tau) d\tau}_{u_I(kT)} + k_I \int_{kT}^{kT+T} e(\tau) d\tau$$

$$u_I(kT + T) \approx u_I(kT) + k_I \frac{T}{2} \left(e(kT + T) + e(kT)\right)$$

The ***trapezoidal rule*** was used for the numerical integration. Other choices for the numerical integration are possible.

Digitization

Emulation: The PID Controller (3)

The *derivative term* is obtained by integrating $u_D(t) = k_D \dot{e}(t)$

$$k_D e(t) = \int_0^t u_D(\tau) d\tau$$

Digitization

Emulation: The PID Controller (3)

The *derivative term* is obtained by integrating $u_D(t) = k_D \dot{e}(t)$

$$k_D e(t) = \int_0^t u_D(\tau) d\tau$$

$$k_D e(kT + T) = \int_0^{kT+T} u_D(\tau) d\tau$$

Digitization

Emulation: The PID Controller (3)

The *derivative term* is obtained by integrating $u_D(t) = k_D \dot{e}(t)$

$$k_D e(t) = \int_0^t u_D(\tau) d\tau$$

$$k_D e(kT + T) = \int_0^{kT+T} u_D(\tau) d\tau$$

$$k_D e(kT + T) = \underbrace{\int_0^{kT} u_D(\tau) d\tau}_{k_D e(kT)} + \int_{kT}^{kT+T} u_D(\tau) d\tau$$

Digitization

Emulation: The PID Controller (3)

The *derivative term* is obtained by integrating $u_D(t) = k_D \dot{e}(t)$

$$k_D e(t) = \int_0^t u_D(\tau) d\tau$$

$$k_D e(kT+T) = \int_0^{kT+T} u_D(\tau) d\tau$$

$$k_D e(kT+T) = \underbrace{\int_0^{kT} u_D(\tau) d\tau}_{k_D e(kT)} + \int_{kT}^{kT+T} u_D(\tau) d\tau$$

$$k_D e(kT+T) \approx k_D e(kT) + \frac{T}{2} \left(u_D(kT+T) + u_D(kT) \right)$$

Digitization

Emulation: The PID Controller (3)

The *derivative term* is obtained by integrating $u_D(t) = k_D \dot{e}(t)$

$$k_D e(t) = \int_0^t u_D(\tau) d\tau$$

$$k_D e(kT+T) = \int_0^{kT+T} u_D(\tau) d\tau$$

$$k_D e(kT+T) = \underbrace{\int_0^{kT} u_D(\tau) d\tau}_{k_D e(kT)} + \int_{kT}^{kT+T} u_D(\tau) d\tau$$

$$k_D e(kT+T) \approx k_D e(kT) + \frac{T}{2}\left(u_D(kT+T) + u_D(kT)\right)$$

$$u_D(kT+T) \approx k_D \frac{2}{T}(e(kT+T) - e(kT)) - u_D(kT)$$

By combining the three terms, the control output of the discrete controller is

$$u(kT+T) = \underbrace{k_P e(kT+T)}_{u_P(kT+T)} + \underbrace{u_I(kT) + k_I \frac{T}{2}\left(e(kT+T) + e(kT)\right)}_{u_I(kT+T)} + \underbrace{k_D \frac{2}{T}\left(e(kT+T) - e(kT)\right) - u_D(kT)}_{u_D(kT+T)}$$

Digitization

Emulation: The PID Controller (3)

By combining the three terms, the control output of the discrete controller is

$$u(kT+T) = \underbrace{k_P e(kT+T)}_{u_P(kT+T)} + \underbrace{u_I(kT) + k_I \frac{T}{2}\left(e(kT+T) + e(kT)\right)}_{u_I(kT+T)} + \underbrace{k_D \frac{2}{T}\left(e(kT+T) - e(kT)\right) - u_D(kT)}_{u_D(kT+T)}$$

Notice that the terms in the controller depends on the sampling time T. Thus, it needs to be known and constant for implementing the PID controller with constant gains.

Digitization

Emulation: The PID Controller (3)

By combining the three terms, the control output of the discrete controller is

$$u(kT+T) = \underbrace{k_P e(kT+T)}_{u_P(kT+T)} + \underbrace{u_I(kT) + k_I \frac{T}{2}\left(e(kT+T) + e(kT)\right)}_{u_I(kT+T)} + \underbrace{k_D \frac{2}{T}(e(kT+T) - e(kT)) - u_D(kT)}_{u_D(kT+T)}$$

Notice that the terms in the controller depends on the sampling time T. Thus, it needs to be known and constant for implementing the PID controller with constant gains.

To analyze the system, the I-term and D-term are z-transformed

$$zu_I(z) = u_I(z) + k_I \frac{T}{2}\left(ze(z) + e(z)\right)$$

$$zu_D(z) = k_D \frac{2}{T}(ze(z) - e(z)) - u_D(z)$$

Digitization

Emulation: The PID Controller (3)

By combining the three terms, the control output of the discrete controller is

$$u(kT+T) = \underbrace{k_P e(kT+T)}_{u_P(kT+T)} + \underbrace{u_I(kT) + k_I \frac{T}{2}\left(e(kT+T) + e(kT)\right)}_{u_I(kT+T)} + \underbrace{k_D \frac{2}{T}(e(kT+T) - e(kT)) - u_D(kT)}_{u_D(kT+T)}$$

Notice that the terms in the controller depends on the sampling time T. Thus, it needs to be known and constant for implementing the PID controller with constant gains.

To analyze the system, the I-term and D-term are z-transformed

$$zu_I(z) = u_I(z) + k_I \frac{T}{2}\left(ze(z) + e(z)\right)$$

$$zu_D(z) = k_D \frac{2}{T}(ze(z) - e(z)) - u_D(z)$$

This gives the following expression for the controller

$$u(z) = \left(k_P + k_I \frac{T}{2}\frac{z+1}{z-1} + k_D \frac{2}{T}\frac{z-1}{z+1}\right) e(z)$$

Digitization

Emulation: The PID Controller (4)

Note that the PID controller in s-domain

$$u(s) = \left(k_P + \frac{k_I}{s} + k_D s\right) e(s)$$

is given in the z-domain as

$$u(z) = \left(k_P + k_I \frac{T}{2} \frac{z+1}{z-1} + k_D \frac{2}{T} \frac{z-1}{z+1}\right) e(z)$$

Digitization

Emulation: The PID Controller (4)

Note that the PID controller in s-domain

$$u(s) = \left(k_P + \frac{k_I}{s} + k_D s\right) e(s)$$

is given in the z-domain as

$$u(z) = \left(k_P + k_I \frac{T}{2}\frac{z+1}{z-1} + k_D \frac{2}{T}\frac{z-1}{z+1}\right) e(z)$$

This means that the discrete controller is obtained by replacing s with (this is called the trapezoidal rule or Tustin's Method)

$$\frac{2}{T}\frac{z-1}{z+1}$$

Digitization

Emulation: The PID Controller (4)

Note that the PID controller in s-domain

$$u(s) = \left(k_P + \frac{k_I}{s} + k_D s\right) e(s)$$

is given in the z-domain as

$$u(z) = \left(k_P + k_I \frac{T}{2}\frac{z+1}{z-1} + k_D \frac{2}{T}\frac{z-1}{z+1}\right) e(z)$$

This means that the discrete controller is obtained by replacing s with (this is called the trapezoidal rule or Tustin's Method)

$$\frac{2}{T}\frac{z-1}{z+1}$$

In conclusion, a discrete equivalent of a controller $K(s)$ is (MATLAB: `c2d`)

$$K_d(z) = K\left(\frac{2}{T}\frac{z-1}{z+1}\right)$$

Digitization

Emulation: Example of PID Controller (1)

Consider the system

$$G(s) = \frac{45}{(s+9)(s+5)}$$

and a PI controller with transfer function

$$K(s) = 1.4\frac{s+6}{s}$$

Digitization

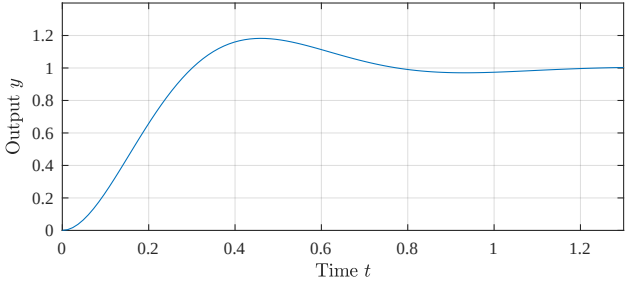
Emulation: Example of PID Controller (1)

Consider the system

$$G(s) = \frac{45}{(s+9)(s+5)}$$

and a PI controller with transfer function

$$K(s) = 1.4\frac{s+6}{s}$$



The system has 20% overshoot and a rise time of 0.2 s.

Digitization

Emulation: Example of PID Controller (2)

A discrete equivalent controller is given by

$$K_d(z) = K\left(\frac{2}{T}\frac{z-1}{z+1}\right) = 1.4\frac{\frac{2}{T}\frac{z-1}{z+1}+6}{\frac{2}{T}\frac{z-1}{z+1}} = 1.4\frac{(1+3T)z+(3T-1)}{z-1}$$

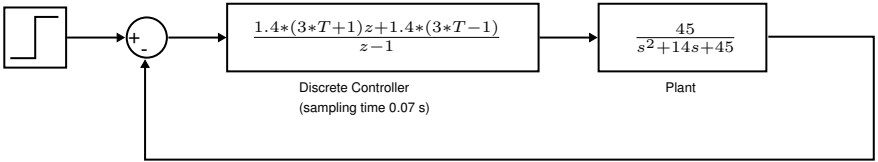
Digitization

Emulation: Example of PID Controller (2)

A discrete equivalent controller is given by

$$K_d(z) = K\left(\frac{2}{T}\frac{z-1}{z+1}\right) = 1.4\frac{\frac{2}{T}\frac{z-1}{z+1}+6}{\frac{2}{T}\frac{z-1}{z+1}} = 1.4\frac{(1+3T)z+(3T-1)}{z-1}$$

A block diagram of the implementation is given below. Note that the sample rate needs to be specified in the boxes.



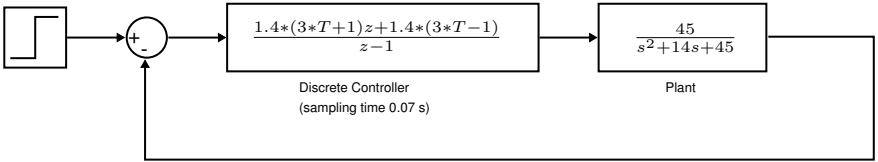
Digitization

Emulation: Example of PID Controller (2)

A discrete equivalent controller is given by

$$K_d(z) = K\left(\frac{2}{T}\frac{z-1}{z+1}\right) = 1.4\frac{\frac{2}{T}\frac{z-1}{z+1}+6}{\frac{2}{T}\frac{z-1}{z+1}} = 1.4\frac{(1+3T)z+(3T-1)}{z-1}$$

A block diagram of the implementation is given below. Note that the sample rate needs to be specified in the boxes.

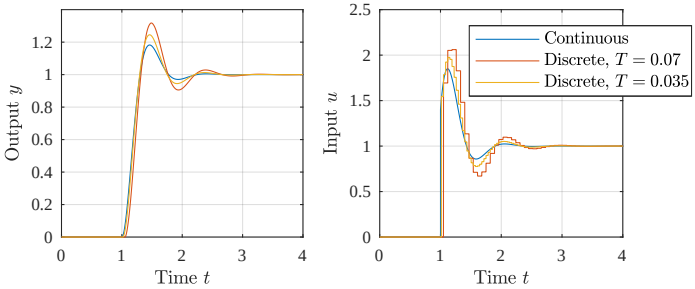


Note that you in practice implement the difference equation - not a discrete transfer function.

Digitization

Emulation: Example of PID Controller (3)

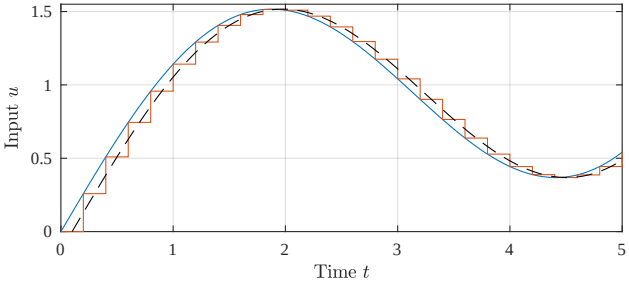
A comparison of controllers with different sampling times show that an decreased sampling time improved the system response.



Digitization

Emulation: Compensation for Sampling Effects (1)

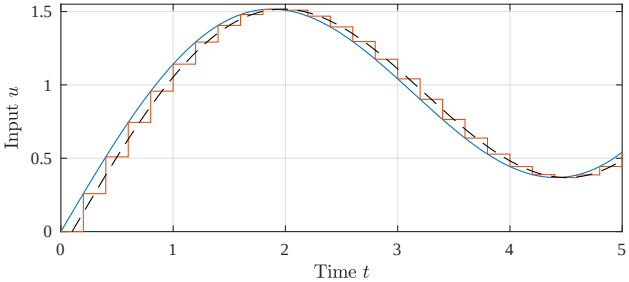
A signal sampled with zero-order hold is in average delayed by $T/2$, where T is the sampling time.



Digitization

Emulation: Compensation for Sampling Effects (1)

A signal sampled with zero-order hold is in average delayed by $T/2$, where T is the sampling time.

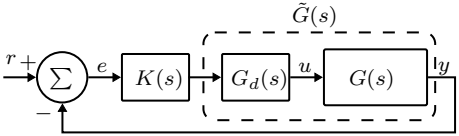


It is appropriate to include this delay in the model when designing a controller by emulation.

Digitization

Emulation: Compensation for Sampling Effects (2)

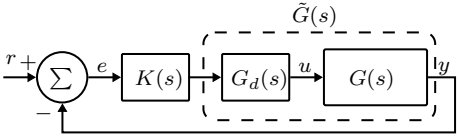
To incorporate the delay caused by zero order hold in the design, the continuous controller should be designed for the system $\tilde{G}(s)$, where $G_d(s)$ models the delay of $T/2$.



Digitization

Emulation: Compensation for Sampling Effects (2)

To incorporate the delay caused by zero order hold in the design, the continuous controller should be designed for the system $\tilde{G}(s)$, where $G_d(s)$ models the delay of $T/2$.



The delay is typically modeled using a first-order system with a time constant of $T/2$, i.e.,

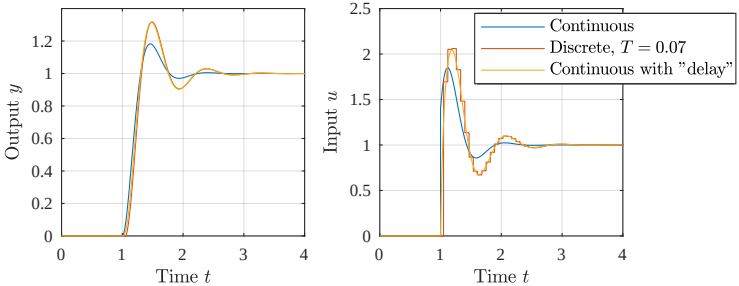
$$G_d(s) = \frac{1}{T/2s + 1}$$

Other models such as Padé approximations may also be used.

Digitization

Emulation: Compensation for Sampling Effects (3)

It is seen that the behavior of the delayed system ($G_d(s)G(s)$) is very close to the behavior of the sampled data system.



Digitization

Emulation: Design Procedure

A discrete controller can be designed by emulation for the system $G(s)$ according to the next procedure.

1. Design continuous compensation for the system $G_d(s)G(s)$, where $G_d(s)$ approximates a delay of $T/2$.
2. Derive the discrete controller by applying Tustin's rule or the matched pole-zero method (other discretization methods exist, but the mentioned methods are preferred).
3. Analyze the design by simulation or experimentally.

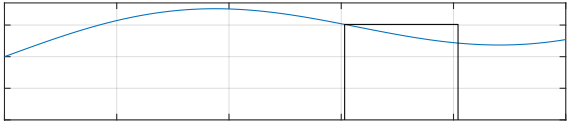
Introduction
Curriculum

Dynamical Systems
Third and Higher Order Systems
Zeros

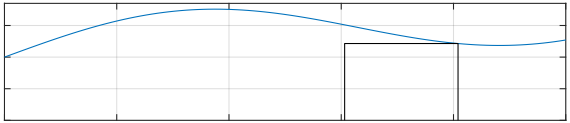
Digitization
Emulation
Numerical Integration Methods
Discrete Design

Summary

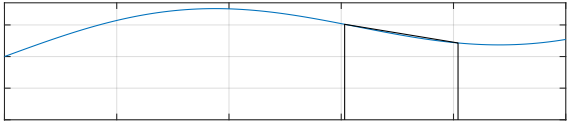
The following integration rules are commonly used.



Forward Rectangular Rule



Backward Rectangular Rule



Trapezoid Rule

Digitization

Transformation

The following rules are used for approximating discrete transfer functions by substituting s by the following expressions

Method	**Approximation**
Forward rule	$s \leftarrow \frac{z-1}{T}$
Backward rule	$s \leftarrow \frac{z-1}{Tz}$
Trapezoid rule	$s \leftarrow \frac{2}{T}\frac{z-1}{z+1}$

Digitization
Transformation

The following rules are used for approximating discrete transfer functions by substituting s by the following expressions

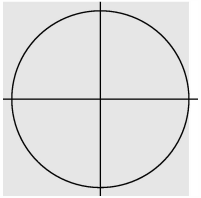
Method	**Approximation**
Forward rule	$s \leftarrow \frac{z-1}{T}$
Backward rule	$s \leftarrow \frac{z-1}{Tz}$
Trapezoid rule	$s \leftarrow \frac{2}{T}\frac{z-1}{z+1}$

To get the transfer function from a discrete transfer function, the variable z is replaced by the following expressions

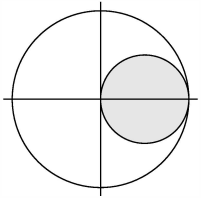
Method	**Approximation**
Forward rectangular rule	$z \leftarrow 1 + Ts$
Backward rectangular rule	$z \leftarrow \frac{1}{1-Ts}$
Bilinear rule	$z \leftarrow \frac{1+Ts/2}{1-Ts/2}$

Digitization

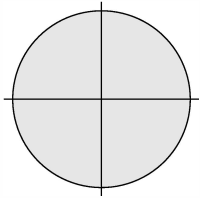
The left half-plane is mapped to different regions of the z-plane (shaded area) dependent on the numeric approximation method.



Forward Rectangular Rule



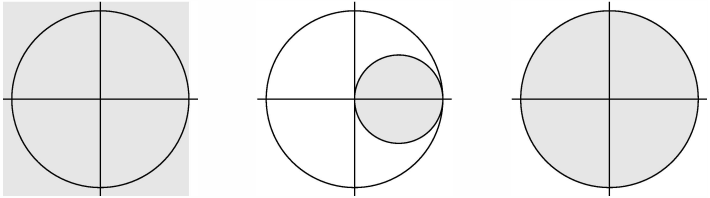
Backward Rectangular Rule



Trapezoid Rule

Digitization

The left half-plane is mapped to different regions of the z-plane (shaded area) dependent on the numeric approximation method.



Forward Rectangular Rule Backward Rectangular Rule Trapezoid Rule

Discretization of a stable system using the forward rectangular rule may lead to an unstable discrete system.

Discrete Design

Introduction
Curriculum

Dynamical Systems
Third and Higher Order Systems
Zeros

Digitization
Emulation
Numerical Integration Methods
Discrete Design

Summary

Discrete Design

Overview

The discrete design method relies on a discretized plant model. The discrete transfer function of a system $G(s)$ and preceding zero-order hold is

$$G(z) = (1 - z^{-1})\mathcal{Z}\left\{\frac{G(s)}{s}\right\}$$

where $\mathcal{Z}$ denotes the z-transformation.

Discrete Design

Overview

The discrete design method relies on a discretized plant model. The discrete transfer function of a system $G(s)$ and preceding zero-order hold is

$$G(z) = (1 - z^{-1})\mathcal{Z}\left\{\frac{G(s)}{s}\right\}$$

where $\mathcal{Z}$ denotes the z-transformation.

Subsequent to the discretization, the control system can be analyzed based on the closed-loop transfer function

$$T_{\mathsf{cl}}(z) = \frac{K(z)G(z)}{1 + K(z)G(z)}$$

Discrete Design

Overview

The discrete design method relies on a discretized plant model. The discrete transfer function of a system $G(s)$ and preceding zero-order hold is

$$G(z) = (1 - z^{-1})\mathcal{Z}\left\{\frac{G(s)}{s}\right\}$$

where $\mathcal{Z}$ denotes the z-transformation.

Subsequent to the discretization, the control system can be analyzed based on the closed-loop transfer function

$$T_{\text{cl}}(z) = \frac{K(z)G(z)}{1 + K(z)G(z)}$$

Since the structure of the continuous and discrete closed-loop transfer functions are the same, the stability of the system can be studied via the characteristic equation

$$1 + K(z)G(z) = 0$$

Discrete Design

Overview

The discrete design method relies on a discretized plant model. The discrete transfer function of a system $G(s)$ and preceding zero-order hold is

$$G(z) = (1 - z^{-1})\mathcal{Z}\left\{\frac{G(s)}{s}\right\}$$

where $\mathcal{Z}$ denotes the z-transformation.

Subsequent to the discretization, the control system can be analyzed based on the closed-loop transfer function

$$T_{\text{cl}}(z) = \frac{K(z)G(z)}{1 + K(z)G(z)}$$

Since the structure of the continuous and discrete closed-loop transfer functions are the same, the stability of the system can be studied via the characteristic equation

$$1 + K(z)G(z) = 0$$

Consequently, the controller $K(z)$ can be designed via root locus or any of the other methods.

Discrete Design

Design Procedure

The following procedure should be followed to designing a discrete controller

1. Transform the continuous-time plant into discrete time as follows

$$G(z) = (1 - z^{-1})\mathcal{Z}\left\{\frac{G(s)}{s}\right\}$$

2. Design the feedback controller $K(z)$ using the same approaches as for a continuous-time.
3. Verify the design on the sample data system.

Discrete Design

Design Example (1)

Consider the following system preceded by a ZOH

$$G(s) = \frac{1}{s^2}$$

Design a controller such that the closed-loop system has a damping ratio $\zeta = 0.7$ and natural frequency $\omega_n = 0.3$ rad/s.

Discrete Design

Design Example (1)

Consider the following system preceded by a ZOH

$$G(s) = \frac{1}{s^2}$$

Design a controller such that the closed-loop system has a damping ratio $\zeta = 0.7$ and natural frequency $\omega_n = 0.3$ rad/s.

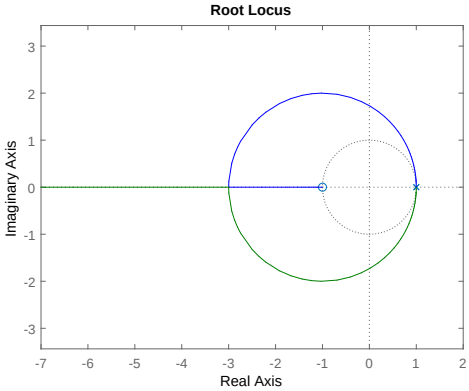
The discrete transfer function is given by

$$G(z) = (1 - z^{-1})\mathcal{Z}\left\{\frac{G(s)}{s}\right\} = (1 - z^{-1})\mathcal{Z}\left\{\frac{1}{s^3}\right\} = (1 - z^{-1})\frac{T^2}{2}\frac{z(z+1)}{(z-1)^3} = \frac{T^2}{2}\frac{z+1}{(z-1)^2}$$

Discrete Design

Design Example (2)

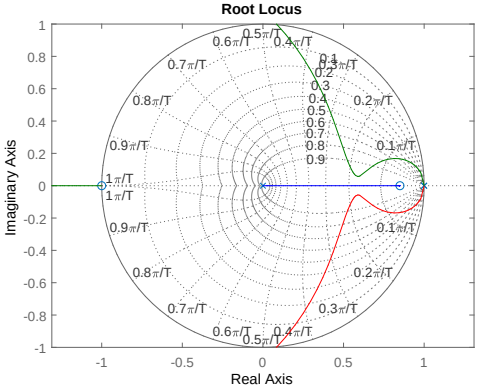
To find a controller, a root locus is generated for a P-controller applied to the system.



Discrete Design

Design Example (3)

A second try for a controller structure is a PD-controller on the form $K(s) = K\frac{z-\alpha}{z}$ with $\alpha = 0.85$.



Summary

Introduction
Curriculum

Dynamical Systems
Third and Higher Order Systems
Zeros

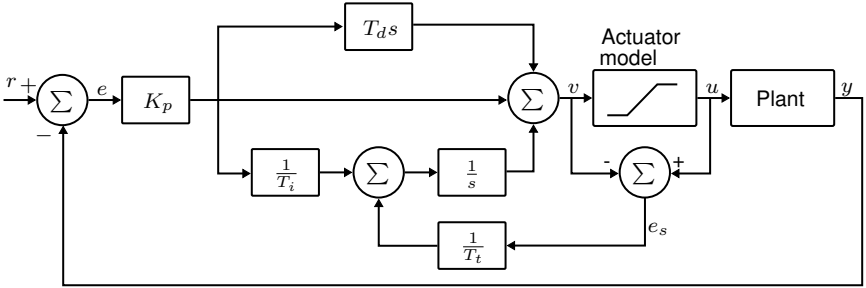
Digitization
Emulation
Numerical Integration Methods
Discrete Design

Summary

Summary

Anti-Windup using Back-Calculation

The anti-windup scheme called ***back-calculation*** recomputes the integral term when the system input is saturated.

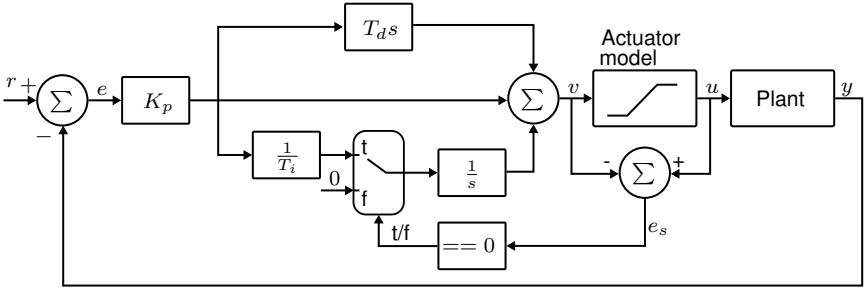


The parameter T_t is called the ***tracking time constant***.

Summary

Anti-Windup using Conditional Integration

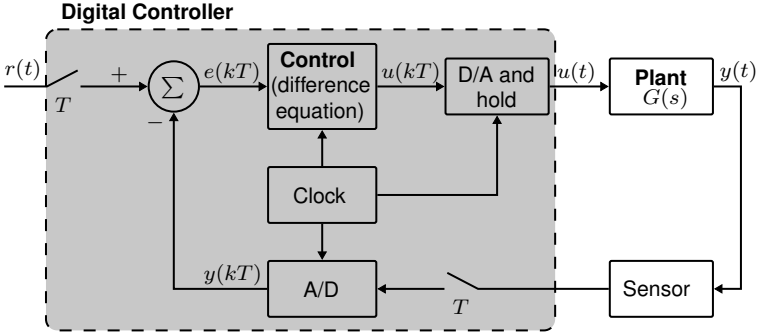
The anti-windup scheme called ***conditional integration*** (also known as clamping) stops the integration, when the system is in saturation.



Summary

System Overview

Sample data systems consisting of both discrete and continuous signals.



The sampling frequency used for the discrete controller should be about 20 times the closed-loop bandwidth.

Summary

Discrete Time PID Controller

The discrete time PID controller is

$$u(kT+T) = \underbrace{k_P e(kT+T)}_{u_P(kT+T)} + \underbrace{u_I(kT) + k_I \frac{T}{2}\left(e(kT+T) + e(kT)\right)}_{u_I(kT+T)} + \underbrace{k_D \frac{2}{T}\left(e(kT+T) - e(kT)\right) - u_D(kT)}_{u_D(kT+T)}$$

The terms in the controller depends on the sampling time T. Thus, it needs to be known and constant for implementing the PID controller with constant gains.

Summary

Design Procedure for Discrete Control by Emulation

A discrete controller can be designed by emulation for the system $G(s)$ according to the next procedure.

1. Design continuous compensation for the system $G_d(s)G(s)$, where $G_d(s)$ approximates a delay of $T/2$.
2. Derive the discrete controller by applying Tustin's rule or the matched pole-zero method (other discretization methods exist, but the mentioned methods are preferred).
3. Analyze the design by simulation or experimentally.

Summary

Design Procedure for Discrete Controller

The following procedure should be followed to designing a discrete controller

1. Transform the continuous-time plant into discrete time as follows

$$G(z) = (1 - z^{-1})\mathcal{Z}\left\{\frac{G(s)}{s}\right\}$$

2. Design the feedback controller $K(z)$ using the same approaches as for a continuous-time.
3. Verify the design on the sample data system.